| | |
|---|---|
| Universidad Galileo | Guatemala 02 de Noviembre del 2026 |
| Facultad: FISICC | **Alumno: Roberto Molina** |
| Curso: Microprocesadores | **Carnet: 24000849** |
| Sección: A | **Hora de Laboratorio: 1:00 a 1:50** |
| Auxiliar: Brandon Mendoza | **Día de Laboratorio: Lunes** |

# Laboratorio #10 Input Capture y UART

## Resumen:

En esta práctica se desarrolló una aplicación para el microcontrolador STM32G431 de la tarjeta NUCLEO-G431KB que combina tres periféricos: un canal de PWM de un timer, un canal de Input Capture de otro timer y el puerto serial UART1. Todo se programó en lenguaje C con CMSIS en Visual Studio Code, y los tres módulos trabajan con interrupciones, sin polling.

Con el PWM se generó una señal rectangular de 5 kHz con un ciclo de trabajo de 50 %, que se aplicó a un circuito RC formado por una resistencia de 82 kΩ y un capacitor de 150 pF, igual que el sensor de humedad HS1101LF de la tarjeta de evaluación. La señal en el capacitor se llevó a la entrada del canal de Input Capture, que está configurado para capturar el flanco de subida en un registro de captura/comparación y el flanco de bajada en otro. Con esos dos valores, dentro de la rutina de interrupción del timer se calculan el período y el ancho de pulso, y a partir de ellos la frecuencia y el ciclo de trabajo.

El UART1 se configuró en los pines PA9 (TX) y PA10 (RX) con 115200 baudios, 8 bits de datos, sin paridad y 1 bit de parada, y con la interrupción de recepción habilitada. Desde una terminal serial en la computadora se envían los caracteres ‘0’, ‘1’, ‘2’ y ‘3’, y la tarjeta responde con el período, la frecuencia, el ancho de pulso o el ciclo de trabajo medidos, según el carácter recibido.

## Objetivos:

- Generar con el PWM de un timer del STM32G431 una señal rectangular de 5 kHz con ciclo de trabajo de 50 %, que sirva de referencia para un circuito RC.
- Configurar un canal de Input Capture con interrupciones para medir el período y el ancho de pulso de la señal en el capacitor, usando el flanco de subida y el de bajada.

- Configurar el puerto serial UART1 con interrupción en la recepción, para comunicar la tarjeta con una terminal serial a través de un cable FTDI.
- Integrar los tres módulos en una aplicación que, según el carácter recibido, muestre en la terminal el período, la frecuencia, el ancho de pulso o el ciclo de trabajo medidos, sin utilizar polling.

# Teoría:

### Comunicación serial asíncrona y UART

La interfaz UART (Universal Asynchronous Receiver-Transmitter) es un puerto serial asíncrono que usa dos líneas, TX para transmitir y RX para recibir, por lo que puede comunicarse en full-duplex [1]. Es asíncrona porque no comparte una línea de reloj: los dos extremos se configuran de antemano con la misma velocidad, en baudios, y cada uno muestrea los bits con su propio reloj [2].

Cada byte viaja en una trama. La línea en reposo está en alto, el transmisor la baja durante un bit (inicio), envía los bits de datos empezando por el menos significativo y termina con uno o más bits de parada en alto [2]. Con la configuración de la práctica, 8 bits de datos, sin paridad y 1 bit de parada (8N1), la trama tiene 10 bits y dura:

$$t_{\text{bit}} = \frac{1}{Baudios}, \quad t_{\text{trama}} = \frac{10}{Baudios}$$

Si los dos extremos no usan los mismos baudios, bits de datos, paridad y bits de parada, la terminal muestra caracteres incorrectos.

### USART1 del STM32G431

El STM32G431 tiene tres USART y dos UART clásicos [1]. En la práctica se usa el USART1 en modo asíncrono, con TX en PA9 y RX en PA10, configurados en función alternativa AF7 [3]. Los registros principales son [1]:

- **USART_BRR:** divisor de la velocidad. Con sobremuestreo por 16 (OVER8 = 0), BRR = f_CK / Baudios.
- **USART_CR1:** UE enciende el USART, RE y TE habilitan el receptor y el transmisor, y RXNEIE genera una interrupción al recibir un dato. Con M0, M1 y PCE en 0 la trama es de 8 bits de datos y sin paridad.
- **USART_CR2:** los bits STOP en 00 dejan 1 bit de parada.

- **USART_ISR, USART_RDR y USART_TDR:** la bandera RXNE indica que hay un dato recibido y se limpia al leer RDR, y TXE indica que se puede escribir otro byte en TDR para transmitir.

**Interrupciones y cable FTDI**

Con polling, el programa espera en un ciclo a que cambie una bandera. Con interrupciones, el periférico avisa al CPU y este solo ejecuta la rutina de servicio cuando hace falta [4]. En el UART, RXNEIE genera una interrupción por cada byte recibido, y además hay que habilitarla en el NVIC con NVIC_EnableIRQ [4]. La rutina debe llamarse exactamente como en la tabla de vectores (por ejemplo, USART1_IRQHandler), y las variables que comparte con el programa principal se declaran volatile para que el compilador siempre lea su valor actual.

Para comunicarse con la computadora se usa un cable FTDI, que convierte USB en un puerto serial con niveles TTL. Se conecta de forma cruzada (TX del cable a PA10 y RX del cable a PA9) y con las tierras unidas, y en la terminal serial se usa la misma configuración: 115200 baudios, 8N1.

**Input Capture**

La función Input Capture guarda el valor del contador (CNT) en un registro de captura (CCRx) en el instante en que ocurre un flanco en el pin del canal [1]. Como la captura la hace el hardware, el instante queda registrado con la resolución del contador aunque el CPU esté ocupado, y la interrupción solo sirve para leer el valor guardado [5]. Los bits principales son [1]:

- **TIMx_CCMR1:** los bits CCxS eligen la entrada del canal. Con 01 (directo) usa su propio pin y con 10 (indirecto) usa el pin del canal vecino.
- **TIMx_CCER:** CCxE habilita la captura, y CCxP y CCxNP eligen el flanco. Con ambos en 0 captura la subida, y con CCxP en 1 y CCxNP en 0, la bajada.
- **TIMx_DIER y TIMx_SR:** CCxIE genera la interrupción y CCxIF indica que hubo una captura. La bandera se limpia al leer CCRx.

Para medir un pulso se capturan los dos flancos de la misma entrada: un canal en modo directo para la subida y otro en modo indirecto para la bajada [1], [5]. El período es la diferencia entre dos subidas consecutivas y el ancho de pulso, la diferencia entre la bajada y la subida. Si C_s es una captura de subida, C_b la de bajada y cada cuenta dura (PSC + 1) / f_CLK:

$$T = (C_s(n) - C_s(n-1)) \cdot \frac{PSC + 1}{f_{CLK}}$$

$$W = (C_b - C_s) \cdot \frac{PSC + 1}{f_{CLK}}$$

$$f = \frac{1}{T}, \quad D = \frac{W}{T} \times 100\ \%$$

Con enteros sin signo, la resta da el resultado correcto aunque el contador haya dado la vuelta entre las dos capturas.

**Circuito RC y sensor de humedad HS1101LF**

El sensor HS1101LF es un capacitor cuyo valor cambia con la humedad relativa. Al conectarlo en serie con una resistencia y aplicarle una señal rectangular, el capacitor se carga y se descarga con una constante de tiempo τ = RC. Por ejemplo, durante la carga su voltaje es:

$$v_C(t) = V_{CC}\left(1 - e^{-t/RC}\right)$$

Si τ es mucho menor que el semiperíodo, el capacitor llega casi a su carga completa en cada ciclo y la señal medida conserva la frecuencia de la referencia, solo con los flancos redondeados. Un cambio en la capacitancia cambia τ y retrasa los flancos que detecta el pin. Si los umbrales de entrada del pin son simétricos respecto a la mitad de la alimentación, el ancho de pulso medido es T/2.

## Resultados prácticos:

```
G4_Test1 > main.c
  1   #include "RTE_Components.h"
  2   #include CMSIS_device_header
  3   #include "USART.h"
  4
  5   void pwm_config(void);
  6   void ic_config(void);
  7   void print_number(uint32_t num);
  8
  9   volatile uint32_t period = 0;
 10   volatile uint32_t width = 0;
 11   uint32_t last_rising_capture = 0; //ultimo valor de captura de flanco de subida
```

*Fig. 1. Definiciones, variables globales y prototipos*

```
void pwm_config(void){
    RCC -> AHB2ENR |= RCC_AHB2ENR_GPIOAEN; //habilitar reloj GPIOA
    RCC -> APB2ENR |= RCC_APB2ENR_TIM1EN; //habilitar reloj TIM1

    GPIOA -> MODER &= ~(3<<16); //limpiar bits 17:16 (PA8)
    GPIOA -> MODER |= (2<<16); //PA8 en modo funcion alterna

    GPIOA -> AFR[1] &= ~(0xF<<0); //limpiar bits 3:0 de AFRH
    GPIOA -> AFR[1] |= (6<<0); //PA8 en AF6 (TIM1_CH1)

    TIM1 -> PSC = 15; //prescaler = 15, 16MHz / (15+1) = 1MHz -> 1 cuenta cada 1us
    TIM1 -> ARR = 199; // 200us = 5kHz
    TIM1 -> CCR1 = 100; //100us en alto 50% duty cycle

    TIM1 -> CCMR1 &= ~((7<<4)|(1<<16)); //limpiar bits OC1M (bits 6:4 y bit 16)
    TIM1 -> CCMR1 |= (6<<4)|(1<<3); //PWM modo 1 salida en 1 mientras CNT<CCR1, OC1PE=1 habilitar preload

    TIM1 -> CCER |= (1<<0); //habilitar salida CH1

    TIM1 -> BDTR |= (1<<15); //habilitar MOE (Main Output Enable)

    TIM1 -> EGR |= (1<<0); //generar evento de actualizacion para cargar PSC, ARR y CCR1 en los registros sombra y reiniciar CNT

    TIM1 -> CR1 |= (1<<7)|(1<<0); //habilitar ARPE y  arranca el contador
}
```

*Fig. 2. Configuración del PWM (TIM1, canal 1)*

```c
void ic_config(void){
    RCC -> AHB2ENR |= RCC_AHB2ENR_GPIOAEN; // habilitar reloj GPIOA
    RCC -> APB1ENR1 |= RCC_APB1ENR1_TIM2EN; // habilitar reloj TIM2

    GPIOA -> MODER &= ~(3<<0); // limpiar bits 1:0 (PA0)
    GPIOA -> MODER |= (2<<0); // PA0 en modo funcion alterna

    GPIOA -> AFR[0] &= ~(0xF<<0); // limpiar bits 3:0 de AFRL
    GPIOA -> AFR[0] |= (1<<0); // PA0 en AF1 (TIM2_CH1)

    TIM2 -> PSC = 15; // prescaler = 15, 16MHz / (15+1) = 1MHz -> 1 cuenta cada 1us
    TIM2 -> ARR = 0xFFFFFFFF; // cuenta libre hasta el maximo valor de 32 bits

    TIM2 -> CCMR1 &= ~((3<<0)|(3<<8)); // limpiar bits CC1S (bits 1:0) y CC2S (bits 9:8)
    TIM2 -> CCMR1 |= (1<<0); // CC1S=01 IC1 conectado a TI1 (PA0)
    TIM2 -> CCMR1 |= (2<<8); // CC2S=10 IC2 conectado a TI1 (PA0)

    TIM2 -> CCER &= ~((1<<1)|(1<<3)); //CC1P=0, CC1NP=0 CH1 captura flanco de subida
    TIM2 -> CCER |= (1<<5); //CC2P=1, CC2NP=0 CH2 captura flanco de bajada
    TIM2 -> CCER |= (1<<0)|(1<<4); // CC1E y CC2E habilitar captura en CH1 y CH2

    TIM2 -> DIER |= (1<<1)|(1<<2); // CC1IE y CC2IE habilitar interrupciones de captura en CH1 y CH2

    TIM2 -> EGR = (1<<0); // generar evento de actualizacion para cargar PSC
    TIM2 -> CR1 |= (1<<0); // CEN arrancar el contador

    NVIC_SetPriority(TIM2_IRQn, 0); // prioridad 0 (mas alta) para no perder flancos
    NVIC_EnableIRQ(TIM2_IRQn); // habilitar interrupcion TIM2 en el NVIC

}
```

*Fig. 3. Configuración del Input Capture (TIM2)*

```c
void TIM2_IRQHandler(void){
    //CC1IF (bit 1): llego flanco de subida
    if(TIM2 -> SR & (1<<1)){
        uint32_t rising_flank = TIM2 -> CCR1; // leer CCR1 limpia CC1IF
        period = rising_flank - last_rising_capture; // subida - subida = periodo
        last_rising_capture = rising_flank; // guardar la ultima captura de flanco de subida
    }

    //CC2IF (bit 2): llego flanco de bajada
    if(TIM2 -> SR & (1<<2)){
        uint32_t falling_flank = TIM2 -> CCR2; // leer CCR2 limpia CC2IF
        width = falling_flank - last_rising_capture; // bajada - subida = ancho de pulso
    }
}
```

*Fig. 4. Rutina de interrupción del Input Capture*

```c
#include "USART.h"

char rx_data[64]; //buffer de linea de recepcion
uint8_t n;
uint8_t i; //indice de escritura en el buffer
volatile char uart_data;   //ultimo caracter recibido por la UART
volatile uint8_t cmd_ready; //1 llego caracter nuevo

void USART_config(uint32_t baudrate){
    //relojes: GPIOA (AHB2) y USART1 (APB2)
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
    RCC->APB2ENR |= RCC_APB2ENR_USART1EN;

    //PA9 y PA10 en modo funcion alterna (MODER = 10)
    GPIOA->MODER &= ~((3 << 18) | (3 << 20));
    GPIOA->MODER |=  ((2 << 18) | (2 << 20));

    //AF7 = USART1 en PA9 (bits 7:4) y PA10 (bits 11:8) de AFRH
    GPIOA->AFR[1] &= ~((0xF << 4) | (0xF << 8));
    GPIOA->AFR[1] |=  ((7 << 4) | (7 << 8));

    //pull-up en PA10 (RX) para que la linea quede en reposo en 1
    GPIOA->PUPDR &= ~(3 << 20);
    GPIOA->PUPDR |=  (1 << 20);

    //8N1 es el valor por defecto (M=00, PCE=0, STOP=00)
    USART1->CR1 = 0;
    USART1->BRR = (SystemCoreClock + baudrate/2) / baudrate; //oversampling x16, redondeado

    //TE (bit 3), RE (bit 2), RXNEIE (bit 5)
    USART1->CR1 |= (1 << 3) | (1 << 2) | (1 << 5);
    //UE (bit 0): habilitar el USART
    USART1->CR1 |= (1 << 0);

    //habilitar interrupcion del USART1 en el NVIC
    NVIC_SetPriority(USART1_IRQn, 1);
    NVIC_EnableIRQ(USART1_IRQn);
}
```

*Fig. 5. Configuración del UART1 (USART_config)*

```c
void USART_send(char c){
    while(!(USART1->ISR & (1 << 7))); //esperar TXE (bit 7): TDR libre
    USART1->TDR = (uint8_t)c;
}

void USART_putString(char * string){
    while(*string){
        USART_send(*string++);
    }
}

void USART1_IRQHandler(void){
    //limpiar overrun ORE (bit 3) si ocurrio, si no la IRQ se queda pegada
    if(USART1->ISR & (1 << 3)){
        USART1->ICR = (1 << 3); //ORECF
    }

    //RXNE (bit 5): llego un dato
    if(USART1->ISR & (1 << 5)){
        uart_data = (char)USART1->RDR; //leer RDR limpia RXNE

        //guardar tambien en el buffer de linea
        if(uart_data == '\r' || uart_data == '\n'){
            rx_data[i] = '\0';
            i = 0;
        } else if(i < sizeof(rx_data) - 1){
            rx_data[i++] = uart_data;
        }

        cmd_ready = 1; //avisar a main()
    }
}
```

*Fig. 6. Transmisión y rutina de interrupción del UART1*

```c
void print_number(uint32_t num){
        char buffer[11]; //uint32 tiene maximo 10 digitos + '\0'
        int index = 10;
        buffer[index] = '\0'; //fin de la cadena

        //llenar el buffer de derecha a izquierda
        do{
            buffer[--index] = '0' + (num % 10); //ultimo digito a ASCII
            num /= 10; //quitar el ultimo digito
        }while(num > 0);

        USART_putString(&buffer[index]); //imprimir desde el primer digito
}
```

*Fig. 7. Función print_number*

```c
int main(void){
    USART_config(115200);
    pwm_config();
    ic_config();

    USART_putString("\n\r  Lab 10 - Input Capture y UART\n\r");
    USART_putString("  0: Periodo  1: Frecuencia  2: Ancho de pulso  3: Duty Cycle\n\r>> ");

    while(1){
        __WFI(); //dormir hasta la siguiente interrupcion

        if(cmd_ready){
            cmd_ready = 0;

            uint32_t p = period; //copia local para que no cambie a mitad del calculo
            uint32_t w = width;

            if(uart_data == '\r' || uart_data == '\n'){
                //ignorar Enter
            }else if(p == 0){
                USART_putString("\n\rNo hay senal en PA0\n\r>> ");
            }else{
                switch(uart_data){
                    case '0':
                        USART_putString("\n\rPeriodo = ");
                        print_number(p);
                        USART_putString(" µs\n\r>> ");
                        break;
                    case '1':
                        USART_putString("\n\rFrecuencia = ");
                        print_number(1000000 / p); //f = 1/T, T en us -> 1000000/T en Hz
                        USART_putString(" Hz\n\r>> ");
                        break;
                    case '2':
                        USART_putString("\n\rAncho de pulso = ");
                        print_number(w);
                        USART_putString(" µs\n\r>> ");
                        break;
                    case '3':
                        USART_putString("\n\rDuty Cycle = ");
                        print_number((w * 100) / p); //duty = ancho/periodo * 100
                        USART_putString("%\n\r>> ");
                        break;
                    default:
                        USART_putString("\n\rComando no valido, use 0-3\n\r>> ");
                        break;
                }
            }
```

*Fig. 8. Programa principal*




*Fig. 9. Terminal serial con las mediciones*

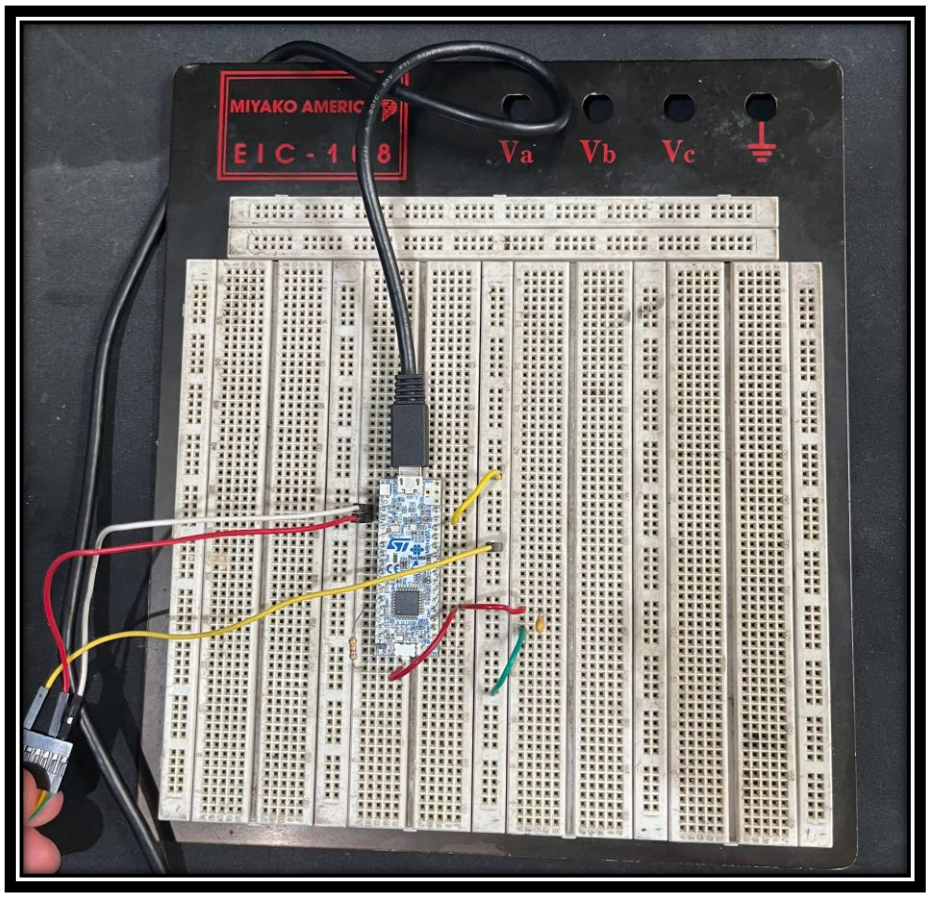



*Fig. 10. Circuito armado*

**Comparación de las mediciones con los valores esperados:**

| Carácter | Salida esperada | Salida obtenida | Observaciones |
|---|---|---|---|
| ‘0’ | Periodo = 200 µs | Periodo = 200 µs | Igual al esperado |
| ‘1’ | Frecuencia = 5000 Hz | Frecuencia = 5000 Hz | Igual al esperado |
| ‘2’ | Ancho de pulso = 100 µs | Ancho de pulso = 101 µs | +1 µs (1 cuenta) |
| ‘3’ | Duty Cycle = 50% | Duty Cycle = 50% | Igual al esperado |
| ‘4’ | Comando no válido | Comando no valido, use 0-3 | Carácter fuera del menú |

**Video demostrativo:** https://youtube.com/shorts/I3LYyOvwC0Y

## Procedimientos teóricos:

**Definiciones y variables globales (Fig. 1)**

Además de RTE_Components.h y el encabezado del dispositivo, se incluyó USART.h, que contiene las funciones del puerto serial. Se declararon los prototipos de pwm_config, ic_config y print_number. Las variables period y width guardan el período y el ancho de pulso en microsegundos, y se declararon volatile porque la rutina de interrupción del timer las modifica y el programa principal las lee. La variable last_rising_capture guarda la última captura del flanco de subida y solo se usa dentro de la interrupción, por lo que no necesita volatile.

**Configuración del PWM (Fig. 2)**

Se usó el canal 1 del TIM1, que sale por el pin PA8. Se habilitó el reloj del puerto A en RCC_AHB2ENR y el del TIM1 con TIM1EN en RCC_APB2ENR. El pin PA8 se puso en función alternativa (10 en MODER) y, como es un pin de 8 a 15, se configuró en AFR[1], donde los bits 3 a 0 se pusieron en 6 (AF6, TIM1_CH1). Con PSC = 15, el contador avanza a:

$$f_{\mathrm{CNT}} = \frac{f_{\mathrm{CLK}}}{PSC + 1} = \frac{16\ MHz}{15 + 1} = 1\ MHz$$

Es decir, una cuenta cada 1 µs. Con ARR = 199 el período tiene 200 cuentas, y con CCR1 = 100 la señal está en alto la mitad del período:

$$f_{\mathrm{PWM}} = \frac{16\,000\,000}{(15 + 1)(199 + 1)} = 5000\ Hz, \quad D = \frac{CCR1}{ARR + 1} = \frac{100}{200} = 50\ \%$$

En CCMR1 se limpiaron los bits OC1M (incluyendo el bit 16) y se escribió 0110 (PWM modo 1), junto con OC1PE para activar el preload. En CCER se habilitó la salida del canal 1 (CC1E) y, como el TIM1 es un timer avanzado, en BDTR se activó MOE, sin lo cual la señal no sale por el pin. Con el bit UG de EGR se cargaron PSC, ARR y CCR1 y, por último, en CR1 se activó ARPE y CEN para arrancar el contador.

**Configuración del Input Capture (Fig. 3)**

Se usó el TIM2, que tiene un contador de 32 bits, con su canal 1 en el pin PA0 (AF1, TIM2_CH1, en AFR[0]). Se usó también PSC = 15, con lo que cada cuenta dura 1 µs. Así, en una señal de 5 kHz se espera una diferencia de 200 cuentas entre dos subidas y de 100 cuentas entre la subida y la bajada. La resolución es de 1 µs, que equivale a 0.5 % del período. ARR se dejó en 0xFFFFFFFF para que el contador cuente libremente, y tarda unos 71.6 minutos en dar la vuelta.

Los canales 1 y 2 se conectaron a la misma entrada TI1 (PA0): CC1S = 01 para el canal 1 (modo directo) y CC2S = 10 para el canal 2 (modo indirecto). En CCER se dejaron CC1P y CC1NP en 0 para que el canal 1 capture el flanco de subida, y se puso CC2P en 1 (con CC2NP en 0) para que el canal 2 capture el flanco de bajada. Se habilitaron las dos capturas con CC1E y CC2E y las interrupciones con CC1IE y CC2IE en DIER. Después se generó un evento de actualización con UG, se encendió el contador con CEN y se habilitó la interrupción en el NVIC con NVIC_EnableIRQ(TIM2_IRQn), con prioridad 0 (la más alta) para no perder flancos.

**Rutina de interrupción del Input Capture (Fig. 4)**

En TIM2_IRQHandler se revisa en SR cuál bandera se activó:

- **CC1IF (bit 1), flanco de subida:** se lee CCR1, lo que limpia la bandera, y se calcula period como la captura actual menos last_rising_capture. Luego esa captura se guarda en last_rising_capture.
- **CC2IF (bit 2), flanco de bajada:** se lee CCR2, que también limpia la bandera, y width se calcula como la captura de bajada menos last_rising_capture, que es la subida anterior.

Como las variables son uint32_t, las restas dan el resultado correcto aunque el contador dé la vuelta. Con las mediciones en microsegundos, se espera obtener T = 200 µs, f = 1 / T = 5 kHz, W = 100 µs y D = W / T × 100 % = 50 %.

**Configuración del UART1 (Figs. 5 y 6)**

El puerto se configura con la función USART_config(baudrate), que en main se llama con 115200. Se habilitó el reloj del puerto A (RCC_AHB2ENR) y el del USART1 (USART1EN en RCC_APB2ENR). Los pines PA9 y PA10 se pusieron en función alternativa (10 en MODER) y en AFR[1] se escribió AF7 en los bits 7 a 4 (PA9) y 11 a 8 (PA10), que es la función del USART1. En PA10 (RX) se activó además la resistencia pull-up, para que la línea quede en 1 cuando está en reposo. Con CR1 en 0 y STOP en 00 en CR2 se dejó la configuración por defecto: 8 bits de datos, sin paridad y 1 bit de parada. El divisor se calcula con el reloj del sistema (16 MHz) y se redondea:

$$BRR = \frac{f_{CK}}{Baudios} = \frac{16\,000\,000}{115\,200} = 138.89 \approx 139$$

Con BRR = 139, la velocidad real y su error son:

$$Baudios\ real = \frac{16\,000\,000}{139} \approx 115\,108, \quad error = \frac{115\,108 - 115\,200}{115\,200} \approx -0.08\,\%$$

El error de −0.08 % es muy pequeño, por lo que la comunicación con la terminal es estable. A esta velocidad cada bit dura 8.68 µs y cada trama de 10 bits, 86.8 µs. Luego se activaron TE, RE y RXNEIE en CR1, se encendió el USART con UE y se habilitó la interrupción en el NVIC con prioridad 1, menor que la del Input Capture (prioridad 0), para que la recepción no haga perder flancos.

Para transmitir (Fig. 6), USART_send espera a que TXE esté en 1 y escribe el byte en TDR, y USART_putString envía una cadena byte por byte. En USART1_IRQHandler primero se limpia la bandera de desbordamiento ORE (con ORECF en ICR) si ocurrió, porque si no la interrupción se queda activa. Luego, si RXNE está activa, se lee RDR, lo que limpia la bandera, y el carácter se guarda en uart_data y en el buffer de línea rx_data (que se cierra con '\0' al recibir Enter). Por último se activa cmd_ready para avisar al programa principal.

**Función print_number (Fig. 7)**

La función convierte un número entero de 32 bits en texto para enviarlo por el UART. Usa un arreglo de 11 caracteres, que alcanza para los 10 dígitos máximos de un uint32_t más el carácter nulo del final. El arreglo se llena de derecha a izquierda: en cada vuelta del ciclo do-while se toma el último dígito con num % 10, se le suma el carácter '0' para convertirlo a ASCII y se elimina con num /= 10. El do-while garantiza que el número 0 también se imprima. Al final, USART_putString envía la cadena desde el primer dígito.

**Programa principal (Fig. 8)**

El programa llama a USART_config, pwm_config e ic_config, imprime el título y el menú de comandos (0: período, 1: frecuencia, 2: ancho de pulso y 3: duty cycle), y entra al ciclo infinito. En el ciclo no se hace polling: con __WFI() el procesador duerme hasta la siguiente interrupción, y solo cuando cmd_ready está activa se atiende el comando recibido. Primero se copian period y width en las variables locales p y w, para que no cambien a la mitad del cálculo. Luego:

- Si el carácter es Enter ('\r' o '\n') se ignora.
- Si p es 0, todavía no se ha medido la señal y se imprime "No hay señal en PA0".
- Con '0' se imprime p, con '2' se imprime w, con '1' se imprime 1 000 000 / p, que es la frecuencia en Hz porque p está en µs, y con '3' se imprime (w · 100) / p, que es el ciclo de trabajo en %. Cualquier otro carácter muestra "Comando no válido, use 0-3".

La frecuencia se muestra en Hz, por lo que 5 kHz aparece como 5000 Hz. El ciclo de trabajo se calcula con división entera, así que se trunca y puede mostrar 49 % cuando el valor real es ligeramente menor a 50 %.

**Valores esperados con el circuito RC**

Con la resistencia de 82 kΩ y el capacitor de 150 pF que se usaron, la constante de tiempo es $\tau = RC = 82\ k\Omega \cdot 150\ pF \approx 12.3\ \mu s$. Como el semiperíodo de la señal de 5 kHz es de 100 µs y $5\tau \approx 61.5\ \mu s$, el capacitor se carga y se descarga casi por completo en cada semiciclo. Por eso la señal que llega a PA0 tiene la misma frecuencia que el PWM, con los flancos redondeados, y se espera medir un período de 200 µs. El ancho de pulso depende de los umbrales de entrada del pin, y es de 100 µs si son simétricos.

**Análisis de resultados (Fig. 9)**

En la terminal se ve el título y el menú de comandos que imprime main al arrancar. Al enviar '0' se obtuvo un período de 200 µs y con '1' una frecuencia de 5000 Hz, que coinciden con lo esperado para la señal de 5 kHz. Con '2' se obtuvo un ancho de pulso de 101 µs, 1 µs más que el valor ideal de 100 µs, y con '3' un ciclo de trabajo de 50 %. El carácter '4', que no está en el menú, mostró el mensaje "Comando no valido, use 0-3", y al volver a enviar '2' se midió otra vez 101 µs.

La diferencia de 1 µs en el ancho de pulso equivale a una cuenta del contador y a 1 % del semiperíodo. Puede deberse a la resolución de 1 µs de la medición y a que los umbrales de entrada del pin PA0 no son exactamente simétricos, lo que desplaza un poco más un flanco que el otro. El ciclo de trabajo se calcula con división entera:

$$D = \frac{w \cdot 100}{p} = \frac{101 \cdot 100}{200} = 50.5 \rightarrow 50\ \%$$

Por eso se muestra 50 % aunque el valor real sea ligeramente mayor. El período y la frecuencia no cambian porque dependen de dos flancos del mismo tipo, y el retraso del circuito RC afecta por igual a ambos.

**Circuito (Fig. 10)**

El pin PA8 (salida del PWM del TIM1) se conecta a la resistencia de 82 kΩ, que va en serie con el capacitor de 150 pF hacia GND, y el punto entre los dos se lleva a PA0 (entrada del Input Capture del TIM2). El pin PA10 (RX) de la tarjeta se conecta al pin TX del cable FTDI, el pin PA9 (TX) al pin RX del cable y el GND del cable a un GND de la tarjeta.

**Prueba del paso 9d**

Al borrar las líneas de TIM2_IRQHandler y volver a cargar el código, la terminal se congeló: ningún carácter enviado obtenía respuesta, como se muestra en el video. Esto ocurre porque las banderas CC1IF y CC2IF solo se limpian al leer CCR1 y CCR2, y esas lecturas estaban dentro de la rutina. Con las banderas activas, la interrupción del TIM2 se vuelve a ejecutar en cuanto termina, una y otra vez, y el CPU queda atrapado en ella.

Como el TIM2 tiene prioridad 0 (la más alta) y el USART1 prioridad 1, la interrupción de recepción del UART nunca llega a atenderse, y el programa principal tampoco vuelve a ejecutarse, por lo que no se procesa ni se responde ningún comando. El PWM, en cambio, sigue saliendo por PA8, porque lo genera el hardware del TIM1 sin intervención del CPU. Esta prueba muestra que en una rutina de interrupción siempre hay que limpiar la bandera que la provocó.

## Conclusiones:

- Fue la primera vez que se utilizó un cable FTDI. Con él se aprendió que su función es convertir el puerto USB de la computadora en un puerto serial con niveles TTL, y que para conectarlo a la tarjeta las líneas van cruzadas: el TX del cable al RX de la tarjeta (PA10), el RX del cable al TX (PA9) y las tierras unidas. Así, la tarjeta se pudo controlar desde una terminal serial, enviando comandos con el teclado y viendo las respuestas en la pantalla.
- También fue la primera vez que se hizo un driver del UART a nivel de registros. Al escribir USART_config, USART_send, USART_putString y la rutina de interrupción se comprendió cómo funciona realmente el puerto serial: cómo se calcula el BRR para obtener 115200 baudios, por qué RX necesita la resistencia pull-up para que la línea quede en reposo en 1 y cómo la interrupción de recepción avisa cada vez que llega un carácter sin que el programa tenga que revisar la bandera. Además, para enviar las mediciones como texto se utilizó la función propia print_number en lugar de sprintf, ya que solo se necesitaba convertir números enteros y así el programa queda más pequeño y sencillo.
- Con el Input Capture se aprendió a medir una señal con el hardware del timer en lugar de hacerlo por software. Se comprendió que, al conectar los dos canales del TIM2 al mismo pin, uno para el flanco de subida y otro para el de bajada, el período y el ancho de pulso se obtienen con solo unas restas en la interrupción. Los valores medidos fueron muy cercanos a los esperados (200 µs, 5000 Hz, 101 µs

y 50 %), y la diferencia de 1 µs en el ancho de pulso mostró que la resolución del contador y el circuito RC también influyen en la medición.

## Bibliografía:


[1] STMicroelectronics, *STM32G4 Series Advanced Arm-Based 32-bit MCUs*, Manual de referencia RM0440. Accedido: 6 nov. 2026. [En línea]. Disponible: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

[2] SparkFun Electronics, "Serial Communication," *SparkFun Learn*. Accedido: 6 nov. 2026. [En línea]. Disponible: https://learn.sparkfun.com/tutorials/serial-communication/all

[3] STMicroelectronics, "STM32G431KB: Mainstream Arm Cortex-M4 MCU," *STMicroelectronics*. Accedido: 6 nov. 2026. [En línea]. Disponible: https://www.st.com/en/microcontrollers-microprocessors/stm32g431kb.html

[4] STMicroelectronics, *STM32 Cortex-M4 MCUs and MPUs Programming Manual*, Manual de programación PM0214. Accedido: 6 nov. 2026. [En línea]. Disponible: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf

[5] STMicroelectronics, "How to use general-purpose timer peripheral on STM32 MCUs," Nota de aplicación AN4776. Accedido: 6 nov. 2026. [En línea]. Disponible: https://www.st.com.cn/resource/en/application_note/an4776-how-to-use-generalpurpose-timer-peripheral-on-stm32-mcus-stmicroelectronics.pdf